

Multi-Label Toxic Comment Classification: Fine-Tuned BERT vs Local LLM (Llama 3.1)

Experimental Report | AIN428 - Information Retrieval

Efe Emirhan Dogan | June 2026

1. Dataset Description

1.1 Source & Size

The dataset originates from the Kaggle Jigsaw Toxic Comment Classification Challenge (data.csv). The raw dataset contains 159,571 comments across 8 columns: id, comment_text, and 6 binary target labels (toxic, severe_toxic, obscene, threat, insult, identity_hate). No missing values were found in any column.

1.2 Original Label Distributions

Label	Count	Density (%)
toxic	15,294	9.59
severe_toxic	1,595	1.00
obscene	8,449	5.30
threat	478	0.30
insult	7,877	4.94
identity_hate	1,405	0.88
Clean (all 0s)	143,346	89.83

The dataset is severely imbalanced. Clean comments constitute 89.83% of the data. The rarest classes are threat (0.30%) and identity_hate (0.88%), posing a significant challenge for minority-class learning.

1.3 Subsampling Strategy

To bypass local hardware constraints (Apple Silicon M1 Unified Memory limits) and drastically reduce computational overhead, a two-stage strategic downsampling was applied:

- Stage 1 - Token Length Filter: Comments exceeding 256 words were removed (~4% of data). This stabilizes sequence lengths and keeps the encoding space uniform (max_length=256).
- Stage 2 - Stratified Knowledge-Dense Sampling: 3,000 comments were randomly sampled from the toxic pool (at least one positive label). 5,000 completely clean comments were sampled. The combined 8,000 rows were shuffled (random_state=42).
- Text Preprocessing: URLs and IP addresses were removed. Newlines and tabs were replaced with spaces. Extra whitespace was collapsed. A cleaned_text column was generated for tokenizer input.

1.4 Label Distributions After Subsampling (8,000 samples)

Label	Count	Density (%)
toxic	2,824	35.30
severe_toxic	312	3.90
obscene	1,586	19.83
threat	98	1.23
insult	1,462	18.28
identity_hate	284	3.55

Multi-Label Toxic Comment Classification: BERT vs Local LLM

Clean (all 0s)	5,000	62.50
----------------	-------	-------

Note: Because a single comment can trigger multiple tags simultaneously, row-wise sums exceed 8,000 (total rows). The clean ratio after subsampling is 62.50% (vs. 89.83% in the original), representing deliberate enrichment of toxic examples.

2. Experimental Design

2.1 5-Fold Cross-Validation Implementation

Standard scikit-learn StratifiedKFold cannot handle multi-label targets. A custom stratification algorithm was developed to preserve the joint label distribution across folds:

- Per-label parsing: For each of the 6 target columns, positive indices were extracted. Each set was partitioned into 5 equal groups via KFold(n_splits=5, shuffle=True, random_state=42 + label_idx).
- Clean sample distribution: Remaining unassigned clean rows (all zeros) were distributed uniformly across 5 folds via separate KFold(n_splits=5, shuffle=True, random_state=99).
- Fold composition: 4 folds (80%, ~5,120 samples) served as training data; 1 fold (20%, ~1,280 samples) was isolated as the validation set.

2.2 Per-Fold Label Distribution

Fold	toxic	sev_toxic	obscene	threat	insult	id_hate
0	473	31	257	12	235	40
1	455	43	266	14	243	39
2	443	44	249	15	240	39
3	442	54	254	12	225	39
4	439	40	249	10	234	39

2.3 Fairness Guarantee: Consistent Test Set & 200-Sample Justification

To preserve absolute experimental fairness, a completely unseen Holdout Test Set of 200 samples was detached at the very beginning of the pipeline using random_state=42 (saved as mini_test_sample.csv). Both the DistilBERT ensemble and Llama 3.1 (zero-shot and few-shot) were forced to run inference against this exact same 200-row arena, eliminating any risk of data leakage or selection bias.

The primary reason for selecting a small test set of only 200 samples is the practical constraint of running Llama 3.1 (8B) locally on consumer hardware. Even with a modest test size of 200 samples, the local LLM inference took approximately 29 minutes for zero-shot (8.69s per sample) and approximately 20 minutes for few-shot (5.92s per sample) evaluation. Running the full test set of 1,600 samples through the LLM pipeline would have required an estimated 3.5-4 hours per experiment, making rapid experimentation and fair comparison infeasible on local Apple Silicon hardware. DistilBERT inference on 200 samples takes only seconds, but to guarantee a fair and balanced comparison, the same 200 samples were used for both models.

3. Fine-Tuned BERT (DistilBERT)

3.1 Model Architecture & Hyperparameters

Parameter	Value
Base Model	distilbert-base-uncased (~66M parameters)
Classification Head	Dropout(p=0.2) + Linear(768, 6)
Hidden Size	768 (from DistilBERT [CLS] token)
Device	Apple Silicon MPS (Metal Performance Shaders)
Batch Size	64
Max Sequence Length	256 tokens (padding + truncation)

Multi-Label Toxic Comment Classification: BERT vs Local LLM

Epochs per Fold	1 (regularization & training time reduction)
Loss Function	BCEWithLogitsLoss (multi-label)
Optimizer	AdamW (lr=2e-5, weight_decay=0.01)
Scheduler	Linear warmup over 10% of training steps
Gradient Clipping	max_norm = 1.0
Inference Threshold	Sigmoid output -> binary at 0.5

A single epoch per fold serves a dual purpose: (1) It acts as a deliberate regularization defense - when fine-tuning heavily pre-trained transformer heads on severely downsampled datasets, multiple epochs cause rapid overfitting on the limited minority class examples (as few as 10 positive threat samples per fold). (2) It significantly reduces total training time - with 5 separate models being trained for 5-fold cross-validation on consumer Apple Silicon (MPS), each additional epoch per fold would multiplicatively increase the already substantial computational budget.

3.2 Ensemble Inference Strategy

During inference on the holdout 200-sample test set, all 5 independent .pt fold weights were sequentially loaded. Their output classification probabilities (sigmoid logits) were averaged via arithmetic mean before the binary threshold of 0.5 was applied. This ensemble approach reduces prediction variance and produces more stable outputs than any individual fold model.

4. Local LLM: Llama 3.1 (8B Instruct)

4.1 Model & Execution Environment

Parameter	Value
Model	Llama 3.1 (8B Instruct)
Execution Framework	Ollama (fully offline)
API Endpoint	http://localhost:11434/api/generate
Temperature	0.0 (deterministic)
Zero-Shot Inference Time	~29 minutes for 200 samples (8.69s/it)
Few-Shot Inference Time	~20 minutes for 200 samples (5.92s/it)

4.2 Zero-Shot Prompt Design

The zero-shot prompt provides only structural category definitions and strict output formatting constraints. The model must rely entirely on its pre-trained internal knowledge. The full system prompt and user prompt used are shown below:

```
System: You are an expert content moderation AI. Analyze the given text and classify it into 6 toxicity categories: toxic, severe_toxic, obscene, threat, insult, and identity_hate. You must respond ONLY with a raw JSON object containing the category names as keys and binary integers (0 or 1) as values. Do not include any explanation, no markdown formatting, no code block backticks, just the raw JSON.
```

```
User: Text to analyze: '{comment_text}'
```

4.3 Few-Shot Prompt Design

7 explicit ground-truth examples were extracted from the source dataset and injected into the system prompt: 1 completely clean comment plus 6 distinct toxic comments, each showcasing a different primary toxicity category. This balanced blueprint aims to define strict decision boundaries without altering the model's internal weights. The full system prompt used is shown below:

```
You are an expert content moderation AI. Analyze the given text and classify it into 6 toxicity categories: toxic, severe_toxic, obscene, threat, insult, and identity_hate.
```

```
Carefully study these real-world examples from the dataset to learn the boundaries of each category:
```

```
Example 1 (Clean):
```

Multi-Label Toxic Comment Classification: BERT vs Local LLM

Text: 'then answer my question why?'
Response: {"toxic": 0, "severe_toxic": 0, "obscene": 0, "threat": 0, "insult": 0, "identity_hate": 0}

Example 2 (Toxic):

Text: 'Why are you guys reverting my edits? Im trying to cooperate with a picture of a native cherokee wedding and you revert it, i dont understand. How about some workout fatty.'
Response: {"toxic": 1, "severe_toxic": 0, "obscene": 0, "threat": 0, "insult": 0, "identity_hate": 0}

Example 3 (Severe Toxic):

Text: 'Go fuckin' hang yourself! Fuckin' scum of the earth! Fuck you asshole!'
Response: {"toxic": 1, "severe_toxic": 1, "obscene": 1, "threat": 0, "insult": 1, "identity_hate": 0}

Example 4 (Obscene):

Text: 'Who is Dreadstar and How is it that they are able to censor me? So I log into Wikipedia one evening and find a few things worth commenting on. Suddenly all my edits are blocked by Dreadstar. WTF and who the heck is Dreadstar?'
Response: {"toxic": 0, "severe_toxic": 0, "obscene": 1, "threat": 0, "insult": 0, "identity_hate": 0}

Example 5 (Threat):

Text: 'YOU ARE A FUCKING NERD. GO KILL YOURSELF YOU NERD.'
Response: {"toxic": 1, "severe_toxic": 1, "obscene": 1, "threat": 1, "insult": 1, "identity_hate": 0}

Example 6 (Insult):

Text: 'Aquaman the Big Stupid Loser This article needs to go into more detail about Aquaman being a loser. I don't care what some fanboys and some dude who traces for a living say how powerful he is. Most people know Aquaman as the loser from the Superfriends who talks to fish.'
Response: {"toxic": 0, "severe_toxic": 0, "obscene": 0, "threat": 0, "insult": 1, "identity_hate": 0}

Example 7 (Identity Hate):

Text: 'All russian are so dumm like you? I mean, you're the best of them. So much to call russia a nation...more like rabbits...'
Response: {"toxic": 1, "severe_toxic": 0, "obscene": 0, "threat": 0, "insult": 1, "identity_hate": 1}

You must respond ONLY with a raw JSON object containing the category names as keys and binary integers (0 or 1) as values. Do not include any explanation.

4.4 Output Parsing Strategy

- Response payload captured from Ollama via `ollama.generate()`.
- Accidental markdown backticks (``json ... ``) are stripped via string replacement.
- Cleaned string is parsed via `json.loads()`.
- `prediction_dict.get(label, 0)` provides safe fallback for absent keys.
- On any exception (network error, invalid JSON), all 6 labels default to 0.

5. Experimental Results

5.1 5-Fold Cross-Validation Summary (Validation Sets)

Fold	Train Loss	Val Loss	Macro F1	Micro F1
0	0.3325	0.1932	0.3791	0.7405
1	0.2923	0.1768	0.3852	0.7494
2	0.2903	0.1754	0.3839	0.7497
3	0.2967	0.1744	0.3972	0.7662
4	0.3061	0.1842	0.3804	0.7409
Mean +/- Std	-	-	0.3852 +/- 0.0064	-

5.2 Per-Fold Test Performance (200-Sample Holdout, DistilBERT Individual Models)

Fold 0

Label	Precision	Recall	F1-Score
toxic	0.8200	0.7069	0.7593
severe_toxic	0.0000	0.0000	0.0000
obscene	0.8182	0.8182	0.8182
threat	0.0000	0.0000	0.0000
insult	0.5217	0.6000	0.5581
identity_hate	0.0000	0.0000	0.0000
Macro F1			0.3559
Micro F1			0.6961

Fold 1

Label	Precision	Recall	F1-Score
toxic	0.8776	0.7414	0.8037
severe_toxic	0.0000	0.0000	0.0000
obscene	0.7500	0.8182	0.7826
threat	0.0000	0.0000	0.0000
insult	0.6000	0.6000	0.6000
identity_hate	0.0000	0.0000	0.0000
Macro F1			0.3644
Micro F1			0.7228

Fold 2

Label	Precision	Recall	F1-Score
toxic	0.8364	0.7931	0.8142
severe_toxic	0.0000	0.0000	0.0000
obscene	0.6923	0.8182	0.7500
threat	0.0000	0.0000	0.0000
insult	0.5500	0.5500	0.5500
identity_hate	0.0000	0.0000	0.0000
Macro F1			0.3524
Micro F1			0.7143

Fold 3

Label	Precision	Recall	F1-Score
toxic	0.8800	0.7586	0.8148
severe_toxic	0.0000	0.0000	0.0000
obscene	0.7727	0.7727	0.7727

Multi-Label Toxic Comment Classification: BERT vs Local LLM

threat	0.0000	0.0000	0.0000
insult	0.6000	0.6000	0.6000
identity_hate	0.0000	0.0000	0.0000
Macro F1			0.3646
Micro F1			0.7264

Fold 4

Label	Precision	Recall	F1-Score
toxic	0.8750	0.7241	0.7925
severe_toxic	0.0000	0.0000	0.0000
obscene	0.7600	0.8636	0.8085
threat	0.0000	0.0000	0.0000
insult	0.5238	0.5500	0.5366
identity_hate	0.0000	0.0000	0.0000
Macro F1			0.3563
Micro F1			0.7094

5.3 Consolidated Final Benchmarks (200-Sample Holdout Test Set)

Metric	DistilBERT Ensemble	Llama 3.1 Zero-Shot	Llama 3.1 Few-Shot
Macro Precision	0.3754	0.2665	0.2560
Macro Recall	0.3628	0.6598	0.5345
Macro F1-Score	0.3685	0.3448	0.3262
Micro Precision	0.7872	0.3276	0.3520
Micro Recall	0.6789	0.8807	0.8073
Micro F1-Score	0.7291	0.4776	0.4903

6. Conclusion & Key Observations

6.1 The Majority Class Dominance (Micro-F1)

DistilBERT heavily outperformed Llama 3.1 in Micro F1-Score (0.7291 vs. 0.4903). Majority classes (toxic, insult, obscene) have abundant semantic representations in the downsampled training partition, enabling the shallow Transformer to successfully optimize decision boundaries for global token distributions. The LLM's high false-positive rate significantly penalized its Micro Precision (0.3276).

6.2 The Power of Zero-Shot Recall

Llama 3.1 demonstrated exceptionally high Micro Recall (0.8807) out-of-the-box. Without seeing a single training sample, its extensive pre-trained world knowledge instantly recognized toxic patterns. However, it exhibited an alarmist behavior (low Precision), flagging slight ironies or minor assertions as toxic. This makes LLMs ideal for recall-critical scenarios (e.g., flagging candidates for human review) but problematic for automated moderation where false positives harm user experience.

6.3 Minority Class Failure in BERT

DistilBERT completely failed to learn three minority classes - severe_toxic, threat, and identity_hate - with F1 = 0.0000 across all 5 folds. These classes had as few as 10-54 positive examples per fold in the training data. The extreme data scarcity, combined with only 1 epoch of training, was insufficient for the model to develop any meaningful decision boundary for these long-tail categories.

6.4 Few-Shot Injection & Safety Alignment Resilience

Balanced ground-truth examples boosted Llama 3.1's global precision and Micro-F1 (0.4776 to 0.4903), but Macro-F1 did not experience a radical transformation (0.3448 to 0.3262). This establishes that the structural safety alignment profiles (RLHF/DPO) burned into modern LLM weights during pre-training act as a permanent, dominant internal prior that resists being overwritten by minor context prompt windows.

6.5 Deployment Tradeoff Summary

Dedicated fine-tuned shallow encoders (DistilBERT) remain the most robust, predictable, and cost-efficient option for precision-critical business constraints where false positives are costly. Large language models (Llama 3.1) provide flexible, training-free baselines with superior recall - ideal for screening and candidate generation where missing a toxic comment is the greater risk. The choice between the two paradigms depends on the error cost asymmetry in the target deployment scenario.